

xiaozhi-esp32 框架技术分析文档

第一模块：系统分析

1.1 项目概述

xiaozhi-esp32 是一个基于 ESP32 平台的开源 AI 聊天机器人项目。该项目旨在帮助开发者理解 AI 硬件开发，将大语言模型应用到实际硬件设备中。

1.2 系统架构

- 硬件层：支持 ESP32-C3、ESP32-S3、ESP32-P4 芯片平台，兼容 70+ 种开源硬件
- 驱动层：包含音频、显示、网络、传感器等硬件驱动
- 协议层：基于 MCP (Message Communication Protocol) 实现设备与云端通信
- 应用层：语音交互、设备控制、系统管理等核心功能
- 服务层：云端 AI 服务集成，支持 Qwen、DeepSeek 等大模型

1.3 核心特性

- 语音交互：基于流式 ASR + LLM + TTS 架构的完整语音交互流水线
- 多协议支持：支持 WebSocket 和 MQTT+UDP 两种通信协议
- 音频编解码：采用 OPUS 音频编解码技术
- 离线唤醒：集成 ESP-SR 离线语音唤醒功能
- 声纹识别：基于 3D Speaker 技术的说话人身份识别
- 多语言支持：支持中文、英文、日文
- OTA 更新：支持固件空中升级
- 多网络连接：支持 Wi-Fi 和 ML307 Cat.1 4G 连接

第二模块：模块目录分析

2.1 项目整体目录结构

项目核心代码位于 main 目录下，其结构清晰，模块划分明确，便于理解和维护。以下是 main 目录的主要组成部分及其模块化介绍：

xiaozhi-esp32/

```
|— main/                                # 核心应用代码
|   |— application.cc/h                  # 应用程序主逻辑（单例模式）
|   |— background_task.cc/h              # 后台任务调度管理
|   |— mcp_server.cc/h                   # MCP 协议服务器实现
|   |— ota.cc/h                           # OTA 固件升级管理
|   |— settings.cc/h                     # 系统配置管理
|   |— system_info.cc/h                  # 系统信息收集
|   |— main.cc                           # 程序入口点
|   |— audio_codecs/                     # 音频编解码器抽象层
|       |— audio_codec.h                 # 音频编解码器基类接口
|       |— es8311_audio_codec.cc/h        # ES8311 音频编解码器实现（单声道音频编解码器，功耗较低，
|       |— es8374_audio_codec.cc/h        # ES8374 音频编解码器实现（支持更高质量的音频处理 主要用于
|       |— es8388_audio_codec.cc/h        # ES8388 音频编解码器实现（主要用于正点原子系列开发板 立体声
|       |— box_audio_codec.cc/h           # ESP-B0X 专用音频编解码器（专门为 ESP-B0X 系列开发板设计
|       |— no_audio_codec.cc/h            # 无音频编解码器实现
|   |— audio_processing/                 # 音频处理模块
|       |— audio_processor.h              # 音频处理器接口
|       |— afe_audio_processor.cc/h        # AFE 音频处理器实现（ESP-AFE 支持实时音频流处理和 VAD
|       |— no_audio_processor.cc/h         # 无音频处理器实现
|       |— wake_word.h                    # 唤醒词检测接口
|       |— afe_wake_word.cc/h              # AFE 唤醒词检测实现（基于 ESP-AFE的唤醒词检测功能 异步处理）
|       |— esp_wake_word.cc/h              # ESP 唤醒词检测实现（基于 ESP-SR 同步检测）
|       |— no_wake_word.cc/h               # 无唤醒词检测实现
|       |— audio_debugger.cc/h             # 音频调试工具
|   |— boards/                           # 硬件板级支持包（70+ 开发板）
|       |— README.md                      # 自定义开发板指南
|       |— common/                         # 通用板级组件
|           |— board.cc/h                  # 板级抽象基类【是所有开发板的根基类，定义所有开发板必须实现的抽
|           |— wifi_board.cc/h             # Wi-Fi 板级基类【Wi-Fi网络连接管理，基于Wi-Fi的网络协议实现
|           |— dual_network_board.cc/h     # 双网络板级基类【支持Wi-Fi和4G网络切换，运行时网络类型
|           |— ml307_board.cc/h            # ML307 4G 板级基类【ML307 4G模块AT指令通信，基于4G的网络协
|           |— esp32_camera.cc/h           # ESP32 摄像头通用实现【ESP32摄像头驱动封装】
|           |— button.cc/h                  # 按钮控制组件【支持GPIO（直接连接到GPIO引脚的物理按钮）和AD
|           |— knob.cc/h                    # 旋钮控制组件【处理旋转编码器输入，双引脚旋转编码器支持，基于
|           |— backlight.cc/h               # 背光控制组件【提供智能背光控制，PWM背光控制实现】（：LCD/OLED
|           |— i2c_device.cc/h              # I2C 设备基类【提供I2C设备通信抽象，电源管理芯片，传感器设备，
|           |— axp2101.cc/h                 # AXP2101 电源管理芯片【电池电量监测】
|           |— sy6970.cc/h                  # SY6970 电源管理芯片
|           |— power_save_timer.cc/h        # 省电定时器【智能省电管理，自动睡眠模式管理】
|           |— system_reset.cc/h            # 系统重置功能
|       |— # ESP32 系列开发板
|       |— bread-compact-esp32/           # 面包板 ESP32 DevKit
```

— bread-compact-esp32-lcd/ # 面包板 ESP32 + LCD
— esp32-cgc/ # ESP32-CGC 开源开发板
— esp32-cgc-144/ # ESP32-CGC-144 开发板
— # ESP32-S3 系列开发板
— bread-compact-wifi/ # 面包板新版接线 (WiFi)
— bread-compact-wifi-lcd/ # 面包板新版接线 + LCD
— bread-compact-wifi-s3cam/ # 面包板新版接线 + 摄像头
— esp-box/ # ESP-B0X 开发板
— esp-box-3/ # ESP-B0X-3 开发板
— esp-box-lite/ # ESP-B0X-Lite 开发板
— esp-s3-lcd-ev-board/ # ESP32-S3-LCD-EV-Board
— esp32s3-korvo2-v3/ # ESP32-S3-Korvo-2 V3
— esp32-s3-touch-amoled-1.8/ # 1.8寸 AMOLED 触摸屏
— esp32-s3-touch-lcd-1.46/ # 1.46寸 LCD 触摸屏
— esp32-s3-touch-lcd-1.85/ # 1.85寸 LCD 触摸屏
— esp32-s3-touch-lcd-1.85c/ # 1.85寸 LCD 触摸屏 (圆形)
— esp32-s3-touch-lcd-3.5/ # 3.5寸 LCD 触摸屏
— m5stack-core-s3/ # M5Stack Core S3
— m5stack-tab5/ # M5Stack Tab5
— atoms3-echo-base/ # AtomS3 + Echo Base
— atoms3r-echo-base/ # AtomS3R + Echo Base
— atoms3r-cam-m12-echo-base/ # AtomS3R CAM/M12 + Echo Base
— atommatrix-echo-base/ # AtomMatrix + Echo Base
— lilygo-t-cameraplus-s3/ # LILYG0 T-CameraPlus S3
— lilygo-t-circle-s3/ # LILYG0 T-Circle S3
— lilygo-t-display-s3-pro-mvsrc-lora/ # LILYG0 T-Display S3 Pro
— sensecap-watcher/ # SenseCAP Watcher
— df-s3-ai-cam/ # DFRobot ESP32-S3 AI 智能摄像头
— doit-s3-aibox/ # 四博智联 AI 陪伴盒子
— waveshare-s3-touch-amoled-1.75/ # 微雪 1.75寸 AMOLED 触摸屏
— lichuang-dev/ # 立创开发板
— atk-dnesp32s3/ # 正点原子 ESP32-S3 开发板
— atk-dnesp32s3-box/ # 正点原子 ESP32-S3 盒子
— atk-dnesp32s3-box0/ # 正点原子 ESP32-S3 盒子 V0
— atk-dnesp32s3m-4g/ # 正点原子 ESP32-S3M 4G 版
— atk-dnesp32s3m-wifi/ # 正点原子 ESP32-S3M WiFi 版
— movecall-cuican-esp32s3/ # Movecall 璀璨 AI 吊坠
— movecall-moji-esp32s3/ # Movecall Moji 小智 AI 衍生版
— tudouzi/ # 土豆子开发板
— taiji-pi-s3/ # 太极派 S3
— esp-sparkbot/ # ESP SparkBot
— esp-spot-s3/ # ESP Spot S3
— electron-bot/ # Electron Bot

```
| | | — otto-robot/          # Otto Robot
| | | — du-chatx/           # DU ChatX
| | | — kevin-box-1/        # Kevin Box 1
| | | — kevin-box-2/        # Kevin Box 2
| | | — kevin-sp-v3-dev/    # Kevin SP V3 开发板
| | | — kevin-sp-v4-dev/    # Kevin SP V4 开发板
| | | — kevin-yuying-313lcd/ # Kevin 语音 313 LCD
| | | — genjutech-s3-1.54tft/ # 根据科技 S3 1.54寸 TFT
| | | — sp-esp32-s3-1.28-box/ # SP ESP32-S3 1.28寸盒子
| | | — sp-esp32-s3-1.54-muma/ # SP ESP32-S3 1.54寸小木马
| | | — zhengchen-1.54tft-wifi/ # 正辰 1.54寸 TFT WiFi
| | | — zhengchen-1.54tft-ml307/ # 正辰 1.54寸 TFT ML307
| | | — xingzhi-cube-0.85tft-wifi/ # 星智魔方 0.85寸 TFT WiFi
| | | — xingzhi-cube-0.85tft-ml307/ # 星智魔方 0.85寸 TFT ML307
| | | — xingzhi-cube-0.96oled-wifi/ # 星智魔方 0.96寸 OLED WiFi
| | | — xingzhi-cube-0.96oled-ml307/ # 星智魔方 0.96寸 OLED ML307
| | | — xingzhi-cube-1.54tft-wifi/ # 星智魔方 1.54寸 TFT WiFi
| | | — xingzhi-cube-1.54tft-ml307/ # 星智魔方 1.54寸 TFT ML307
| | | — minsi-k08-dual/     # 敏思 K08 双网络版
| | | — mixgo-nova/         # MixGo Nova
| | | — # ESP32-C3 系列开发板
| | | — esp-hi/             # ESP-Hi 超低成本 AI 对话机器人
| | | — kevin-c3/           # Kevin C3
| | | — lichuang-c3-dev/    # 立创 C3 开发板
| | | — magiclick-c3/       # Magiclick C3
| | | — magiclick-c3-v2/    # Magiclick C3 V2
| | | — xmini-c3/           # XMini C3
| | | — # ESP32-C6 系列开发板
| | | — waveshare-c6-lcd-1.69/ # 微雪 ESP32-C6-LCD-1.69
| | | — # ESP32-P4 系列开发板
| | | — waveshare-p4-nano/   # 微雪 ESP32-P4-Nano
| | | — waveshare-p4-wifi6-touch-lcd-4b/ # 微雪 P4 WiFi6 触摸 LCD 4B
| | | — waveshare-p4-wifi6-touch-lcd-xc/ # 微雪 P4 WiFi6 触摸 LCD XC
| | | — # 4G/蜂窝网络开发板
| | | — bread-compact-ml307/ # 面包板新版接线 (ML307 AT)
| | | — df-k10/             # DFRobot K10 4G 开发板
| | | — magiclick-2p4/       # Magiclick 2.4G
| | | — magiclick-2p5/       # Magiclick 2.5G
| — display/                # 显示驱动抽象层
| | — display.cc/h          # 显示设备基类接口
| | — oled_display.cc/h     # OLED 显示驱动 (SSD1306等) 【I2C/SPI接口：支持常见的OLED通信
| | — spi_lcd_display.cc/h  # SPI LCD 显示驱动 【小尺寸彩屏：1.3"、1.8"、2.4"等SPI LCD
| | — rgb_lcd_display.cc/h  # RGB LCD 显示驱动 【尺寸屏幕：适合3.5"、4.3"、7"等大屏 GC9
```

	no_display.cc/h	# 无显示设备实现
	lvgl_port.cc/h	# LVGL 图形库移植层 【提供丰富的UI组件和动画】
	iot/	# IoT 设备管理（已弃用，推荐使用 MCP）
	thing_manager.cc/h	# 设备状态管理器
	thing.cc/h	# IoT 设备抽象基类
	README.md	# IoT 模块使用说明
	led/	# LED 控制模块
	led.cc/h	# LED 控制基类
	ws2812_led.cc/h	# WS2812 RGB LED 驱动
	single_led.cc/h	# 单色 LED 驱动
	no_led.cc/h	# 无 LED 实现
	protocols/	# 通信协议实现
	protocol.cc/h	# 协议抽象基类
	mqtt_protocol.cc/h	# MQTT+UDP 协议实现 【MQTT用于控制消息 + UDP用于音频数据传输】
	websocket_protocol.cc/h	# WebSocket 协议实现 【WebSocket同时处理控制消息和音频数据】
	http_client.cc/h	# HTTP 客户端实现
	udp_client.cc/h	# UDP 客户端实现
	binary_protocol.h	# 二进制协议定义
	assets/	# 多语言资源文件
	common/	# 通用音效文件
	zh-CN/	# 中文简体语音资源
	zh-TW/	# 中文繁体语音资源
	en-US/	# 英文语音资源
	ja-JP/	# 日文语音资源
	lang_config.h	# 语言配置头文件
	CMakeLists.txt	# 主模块构建配置 【定义"如何编译" - 源文件组织和编译规则】
	Kconfig.projbuild	# 项目配置选项（70+ 开发板选择） 【定义"编译什么" - 功能选择和树莓派板选择】
	idf_component.yml	# ESP-IDF 组件依赖 【定义"依赖什么" - 外部组件和库管理】
	components/	# 自定义组件库
	docs/	# 项目文档
	v1/	# 版本 1 文档
	websocket.md	# WebSocket 协议文档
	tools/	# 开发工具
	CMakeLists.txt	# 顶层构建文件
	sdkconfig.defaults	# SDK 默认配置
	README.md	# 项目说明（中文）
	README_en.md	# 项目说明（英文）
	README_ja.md	# 项目说明（日文）

唤醒对比

特性	EspWakeWord	AfeWakeWord
处理模式	同步检测	异步处理
任务数量	单线程	多任务
音频预处理	无	AFE 全套处理
数据缓存	无	2秒音频缓存
音频编码	不支持	完整 Opus 编码
内存优化	基础	PSRAM/SPIRAM 优化
多通道支持	基础	完整多通道配置
回声消除	无	支持 AEC

自定义开发板指南的具体实现

```
boards/
├── board-name-1/
│   ├── board_name_1.cc      # 实现文件
│   ├── config.h             # 硬件配置
│   └── config.json          # 编译配置
├── board-name-2/
└── ...
```

- 1. 创建新开发板目录
mkdir main/boards/my-custom-board
- 2. 定义硬件配置
创建 config.h 文件，定义所有GPIO映射和硬件参数：
 - 音频I2S引脚配置
 - 显示屏SPI/QSPI配置
 - 按钮和LED引脚
 - I2C设备地址和引脚
 - 电源管理配置
- 3. 配置编译参数
创建 config.json 文件，指定：
 - 目标芯片型号（esp32/esp32s3/esp32c3等）

- 开发板唯一名称
- 特殊编译选项（Flash大小、分区表等）

4. 实现板级初始化代码

创建 xxx_board.cc 文件，实现：

- 继承合适的基类（WifiBoard 【WIFI版】 /MI307Board 【4G模块基类】 /DualNetworkBoard 【支持Wi-Fi和4G网络切换:】）
- 初始化所有硬件组件
- 重写虚函数提供硬件抽象接口
- 使用 DECLARE_BOARD 宏注册开发板

5. 支持的硬件组件

框架支持多种硬件组件的抽象：

显示屏驱动：

- ST7789 (SPI)
- ILI9341 (SPI)
- SH8601 (QSPI)
- GC9503 等

音频编解码器：

- ES8311（常用）
- ES7210（麦克风阵列）
- AW88298（功放）

电源管理：

- AXP2101
- SY6970

IoT设备控制：

- Speaker（扬声器控制）
- Screen（屏幕亮度调节）
- Battery（电池电量读取）
- Light（灯光控制）

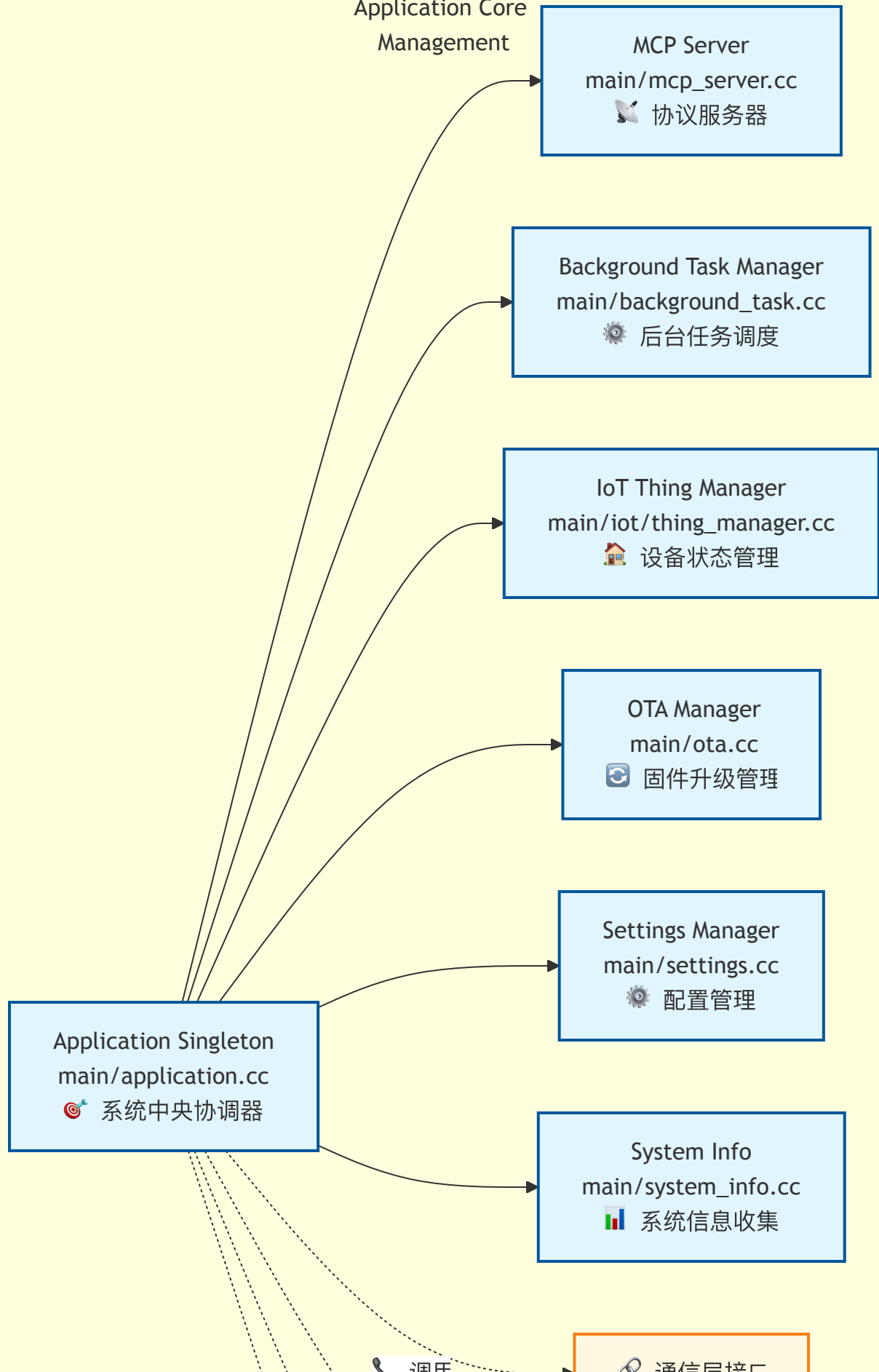
2.2 系统架构流程图

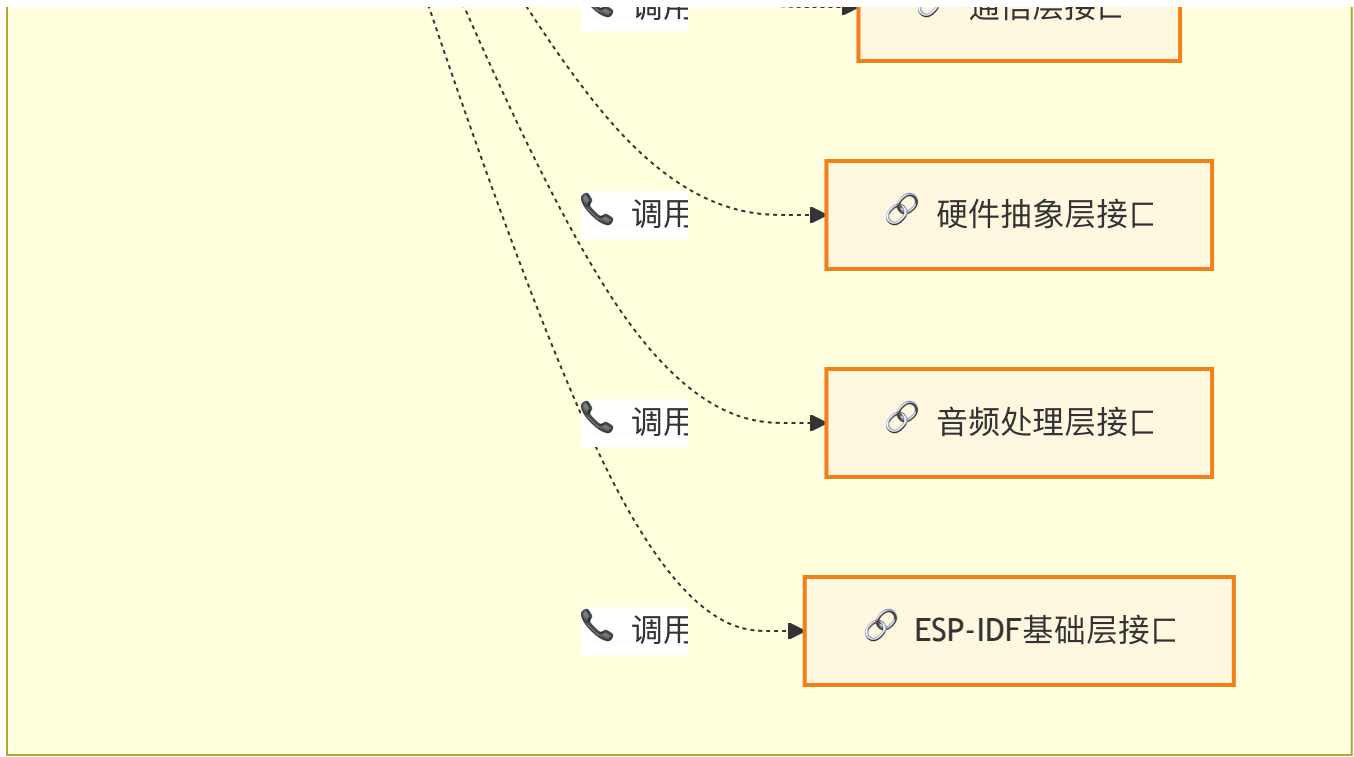
2.2.1 整体系统架构图

模块化架构图表设计

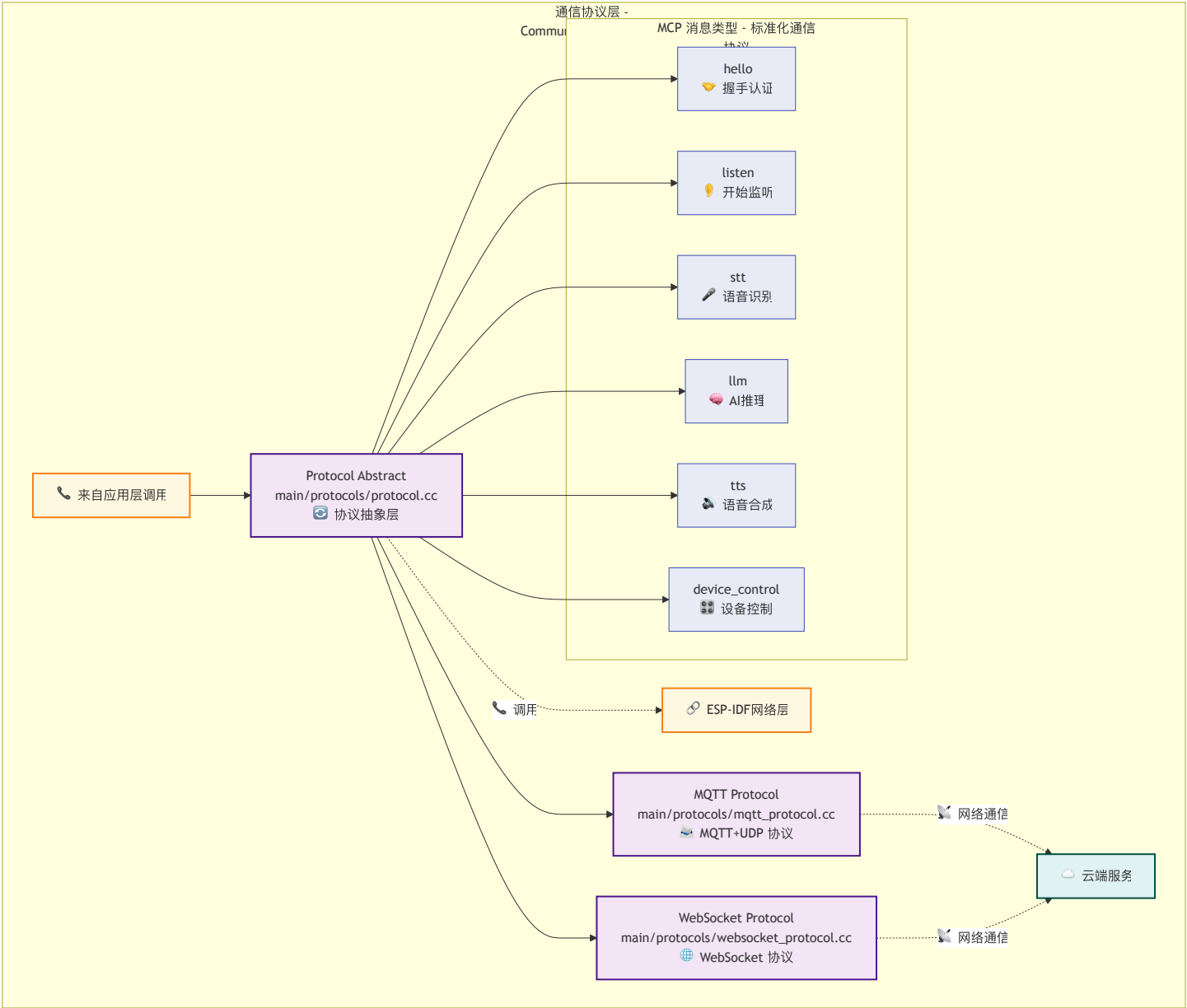
模块1：应用层核心管理架构

应用层核心管理 -
Application Core
Management

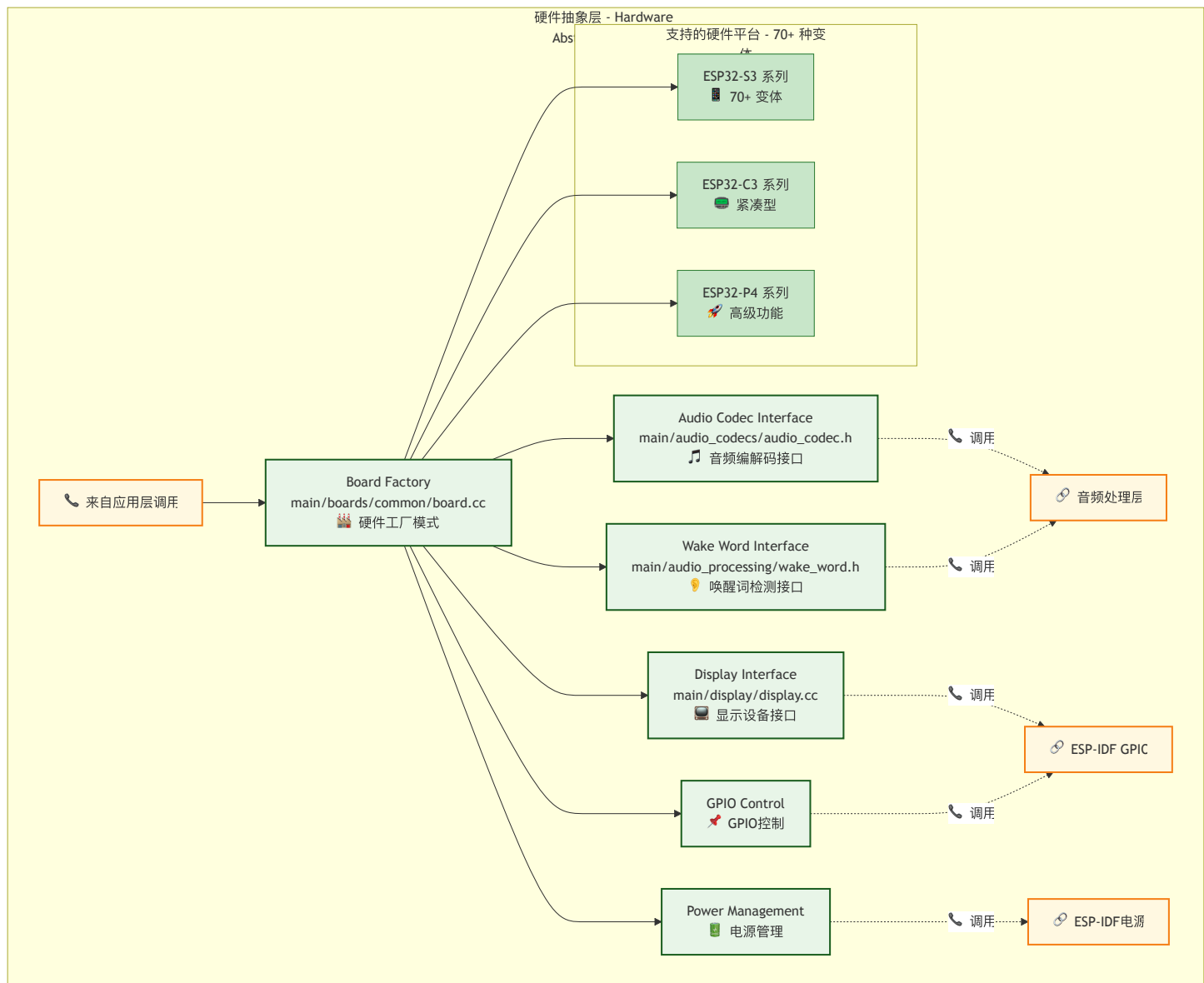




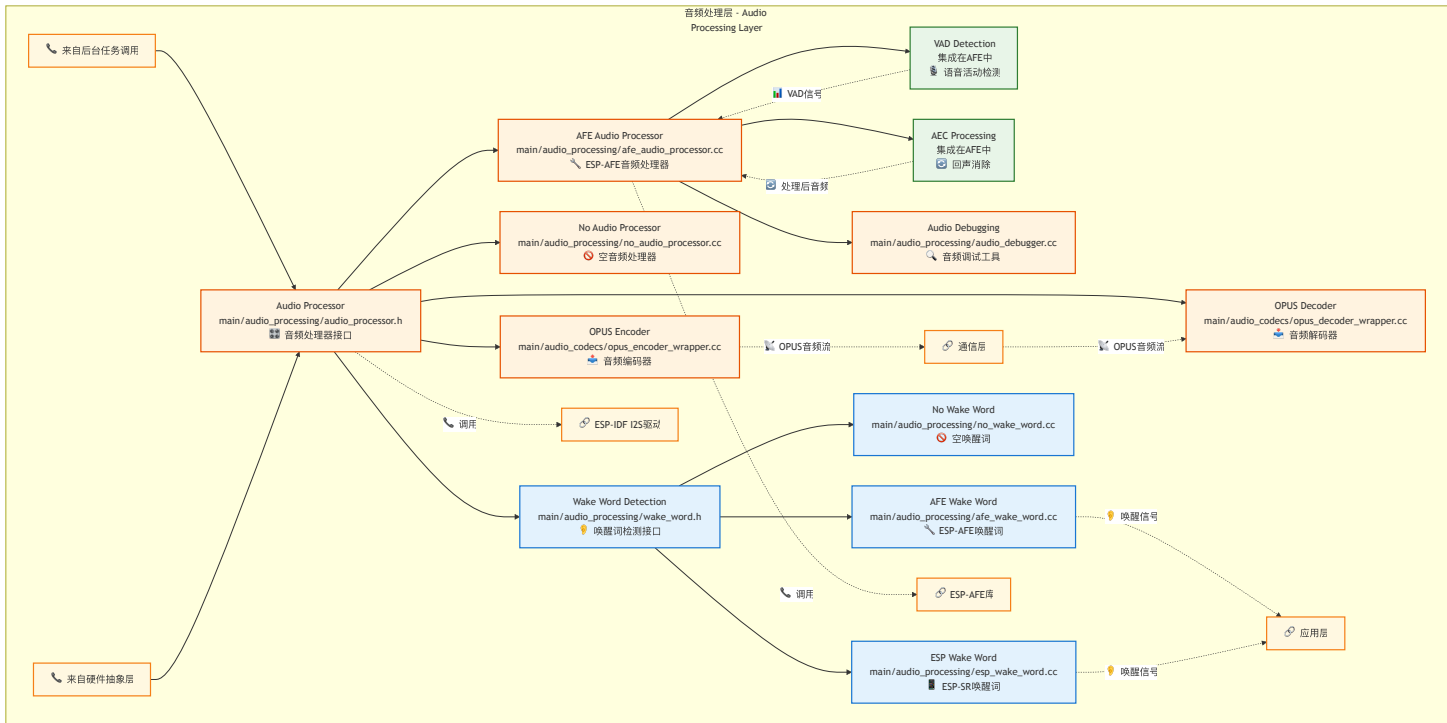
模块2：通信协议架构



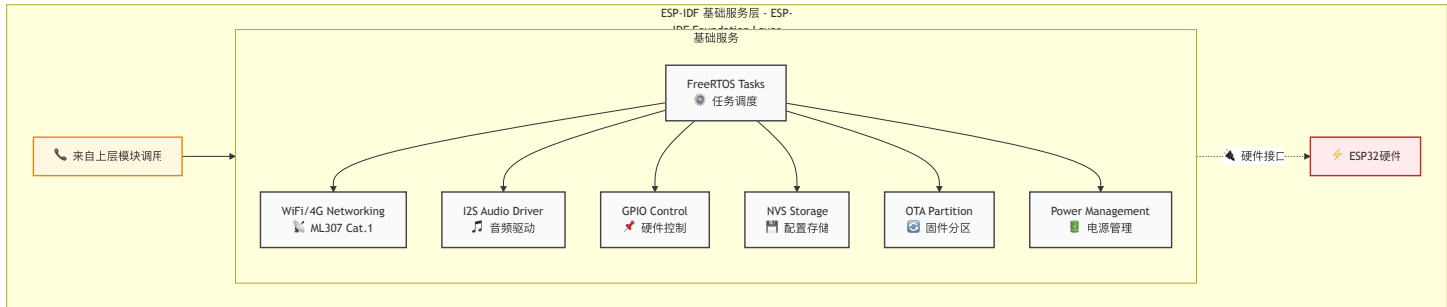
模块3：硬件抽象层架构



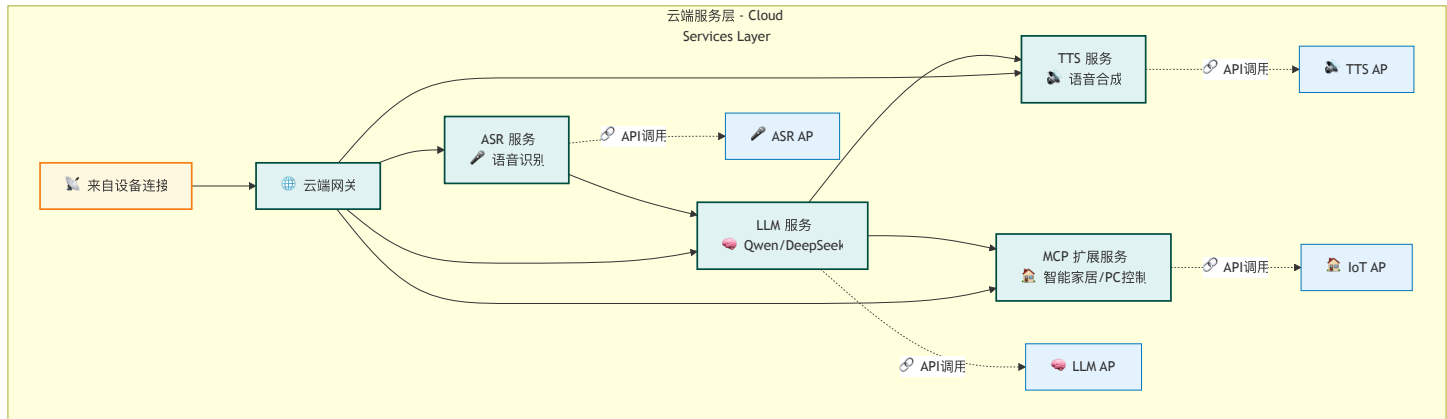
模块4：音频处理层架构



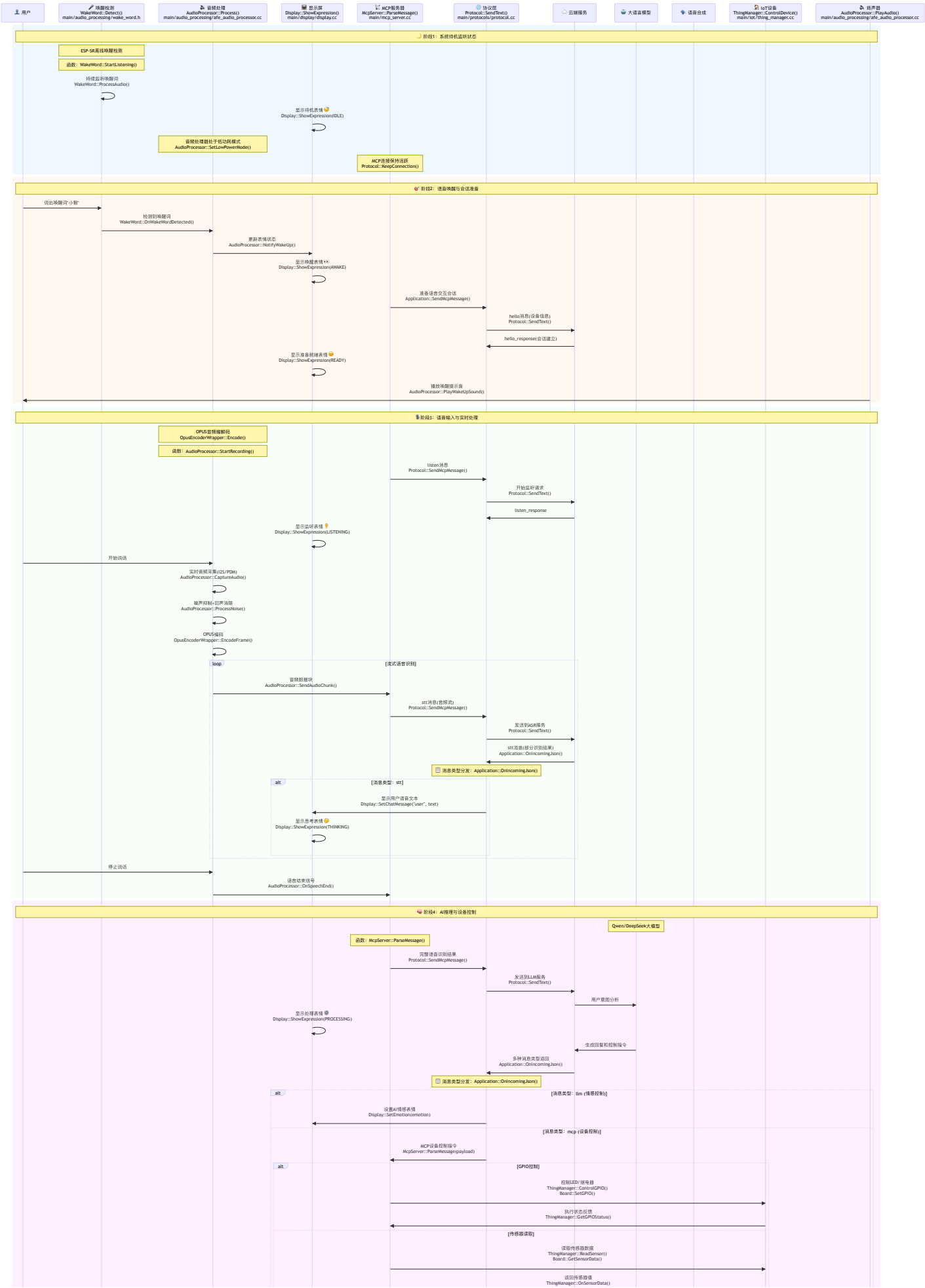
模块5：ESP-IDF基础服务层架构

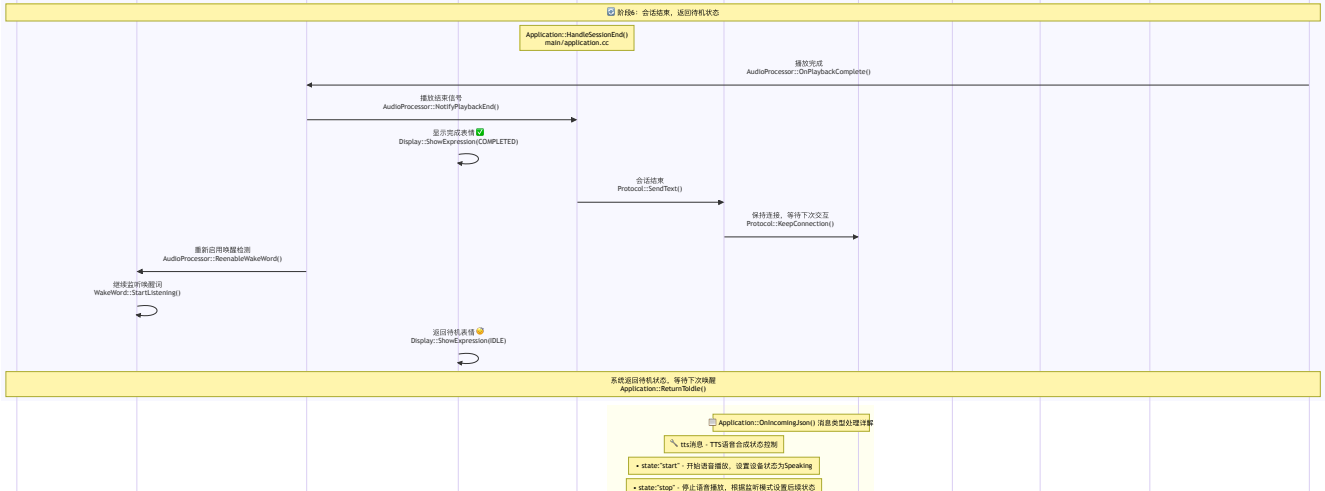
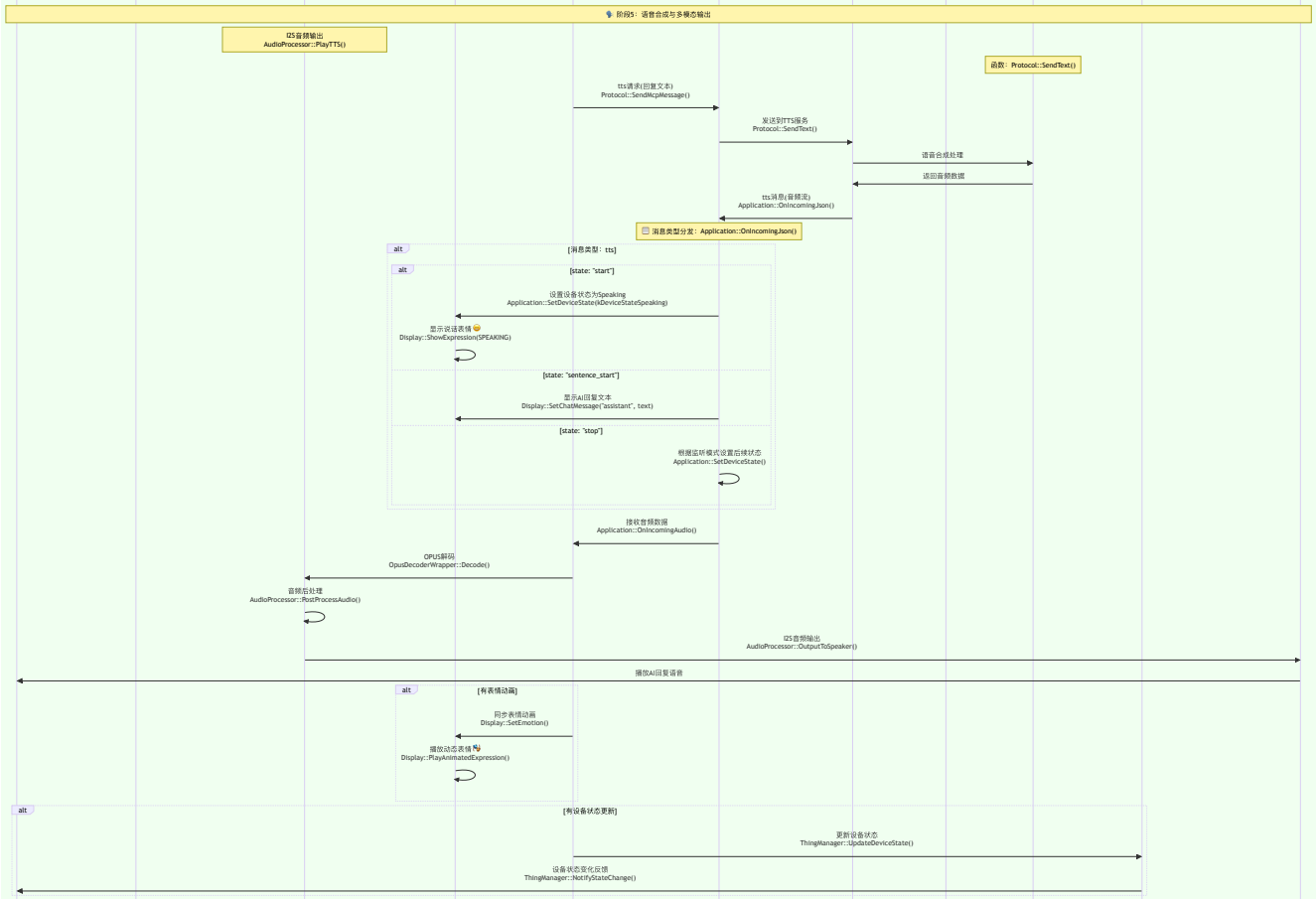
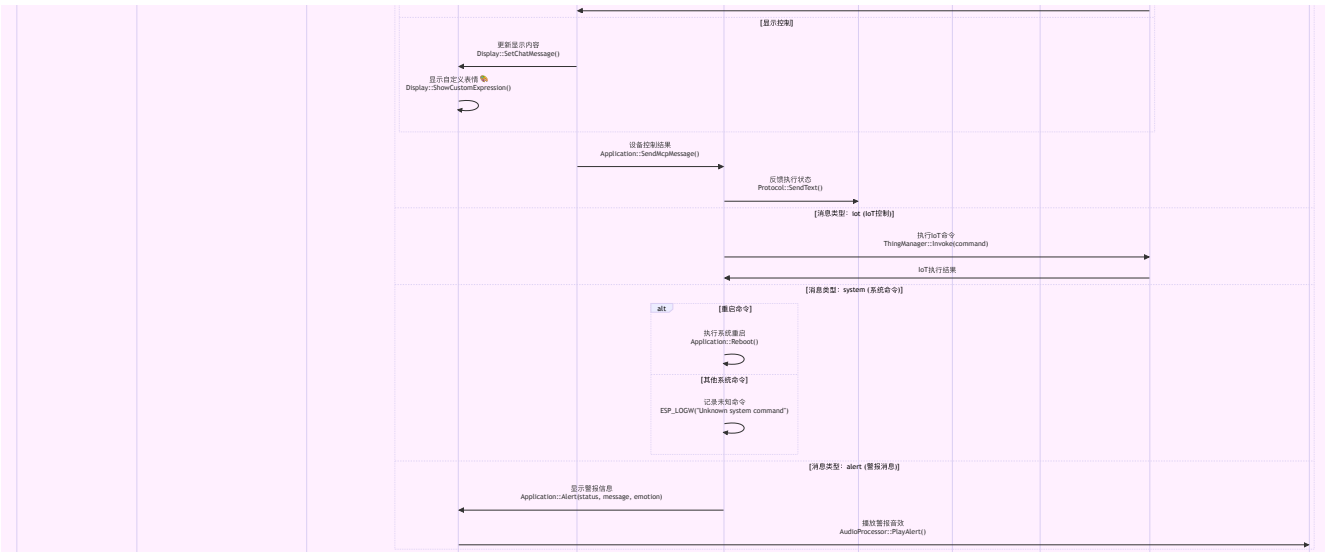


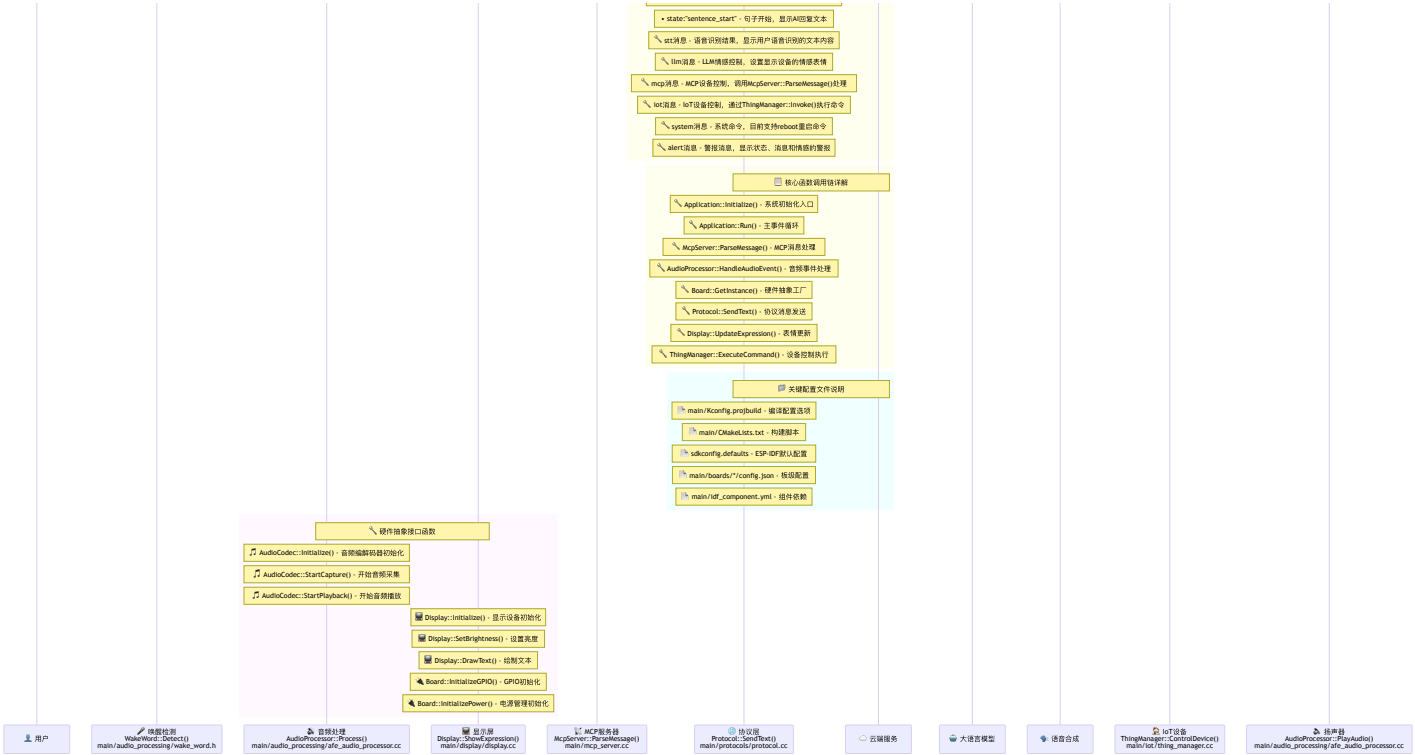
模块6：云端服务架构



综合系统运行流程图







第三模块：模块目录功能介绍